

Ασφάλεια Συστημάτων Υπολογιστών 2024-2025

ΘΕΜΑ 1

“Υλοποίηση TLS, IPSEC, Tor”

Γεώργιος Τουμαράς
58420

Γεώργιος Αγακεχαγιάς
58508

ΦΑΣΗ 1

ΥΛΟΠΟΙΗΣΗ ΤΗΣ ΕΠΙΚΟΙΝΩΝΙΑΣ ΤΩΝ ΧΡΗΣΤΩΝ

Στόχος του μέρους 1 της εργασίας είναι η υλοποίηση μια εφαρμογής η οποία θα επιτρέπει την επικοινωνία μεταξύ δύο χρηστών σε επίπεδο εφαρμογής sockets.

Κώδικας server_phase_1

Ο κώδικας υλοποιεί έναν απλό server που επιτρέπει την αμφίδρομη επικοινωνία μεταξύ του server και ενός client μέσω δικτύου χρησιμοποιώντας το πρωτόκολλο TCP. Ο server δέχεται συνδέσεις από clients, λαμβάνει μηνύματα από αυτούς και μπορεί να στείλει μηνύματα πίσω. Δημιουργούμε ένα αντικείμενο socket το οποίο θα χρησιμοποιηθεί για τη σύνδεση.

```
server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
```

Εδώ ορίζεται ότι θα χρησιμοποιηθεί το πρωτόκολλο IPv4 και με το socket.SOCK_STREAM υποδεικνύουμε ότι θα χρησιμοποιήσουμε το πρωτόκολλο TCP.

```
server_socket.bind(("0.0.0.0", 12345))
```

Δεσμούμε τον server στην διεύθυνση 0.0.0.0 που σημαίνει ότι είναι προσβάσιμος από οποιαδήποτε διεύθυνση IP και στην θύρα 12345 όπου θα είναι το port που θα συνδέονται οι δύο υπολογιστές.

```
def receive_messages(client_socket):
```

Αυτή η συνάρτηση εκτελείται σε ένα ξεχωριστό thread για κάθε client. Χρησιμοποιεί client_socket.recv(1024) για να λαμβάνει δεδομένα από τον client.

Αν τα δεδομένα είναι άδεια (not data), σημαίνει ότι ο client αποσυνδέθηκε.

Εμφανίζει τα δεδομένα που λαμβάνει στην κονσόλα.

```
def send_messages(client_socket):
```

Αυτή η συνάρτηση εκτελείται σε άλλο thread.

Χρησιμοποιεί input() για να διαβάσει μηνύματα από τον χρήστη του server.

Στέλνει τα μηνύματα στον client με τη μέθοδο client_socket.send().

```
while True:
    client_socket, addr = server_socket.accept()
```

Ο server περιμένει μέχρι να συνδεθεί ένας client.

Η μέθοδος accept() επιστρέφει δύο τιμές:

client_socket: Ένα νέο socket για την επικοινωνία με τον συγκεκριμένο client.

addr: Η διεύθυνση (IP, θύρα) του client.

```
threading.Thread(target=receive_messages, args=(client_socket,)).start()
threading.Thread(target=send_messages, args=(client_socket,)).start()
```

Μετά τη σύνδεση, δημιουργούνται δύο threads ένα για τη λήψη μηνυμάτων από τον client και ένα για την αποστολή μηνυμάτων στον client. Η χρήση threads εξασφαλίζει την παράλληλη εκτέλεση για λήψη και αποστολή δεδομένων, βελτιώνοντας την απόδοση.

Κωδικας client_phase_1

Ο κώδικας αυτός υλοποιεί έναν client που συνδέεται με το server_phase_1 μέσω του πρωτοκόλλου TCP. Ο client μπορεί να στέλνει και να λαμβάνει μηνύματα ταυτόχρονα, χρησιμοποιώντας threads.

```
client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
```

Ομοίως με τον server_phase_1 ορίζουμε ένα socket τύπου TCP (SOCK_STREAM) και IPv4 (AF_INET). Αυτό επιτρέπει την αξιόπιστη μεταφορά δεδομένων (με έλεγχο σφαλμάτων) μέσω σύνδεσης.

```
client_socket.connect(("192.168.1.84", 12345))  
print("🔒 Συνδέθηκε με επιτυχία στον Server!")
```

Ο client προσπαθεί να συνδεθεί με την διεύθυνση IP του server (192.168.1.84) και θύρα 12345. Αν η σύνδεση επιτύχει, εμφανίζεται σχετικό μήνυμα.

```
def receive_messages():
```

Αυτή η συνάρτηση ακούει συνεχώς για εισερχόμενα μηνύματα από τον server. Η μέθοδος recv(1024) λαμβάνει μέχρι 1024 bytes. Αν προκύψει σφάλμα, εμφανίζεται μήνυμα και τερματίζεται η λήψη.

```
def send_messages():
```

Η συνάρτηση αυτή καλείται για να στείλει μήνυμα στον server. Ο client εισάγει μήνυμα μέσω της input. Αν πληκτρολογήσει exit, το μήνυμα στέλνεται στον server και η σύνδεση κλείνει.

```
threading.Thread(target=receive_messages).start()  
threading.Thread(target=send_messages).start()
```

Δημιουργούνται δύο threads:

Ένα για λήψη μηνυμάτων (receive_messages).

Ένα για αποστολή μηνυμάτων (send_messages).

Έτσι, ο client μπορεί να στέλνει και να λαμβάνει μηνύματα ταυτόχρονα, χωρίς καθυστερήσεις.

Σύνδεση μεταξύ server_phase_1 και client_phase_1

Ο υπολογιστής που τρέχει τον server_phase_1 έχει την IP 192.168.1.84 η οποία μπορεί να αλλάξει. Για να γίνει σύνδεση μεταξύ client και server προϋποθέτει στον κώδικα του client να εισάγουμε στο client_socket.connect την σωστή IP του χρήστη η οποία βρίσκεται αν ανοίξουμε το cmd και να πληκτρολογήσουμε ipconfig. Αφού κάνουμε αυτή την αλλαγή μπορούμε να τρέξουμε τα προγράμματα. Πρώτα πρέπει να συνδεθεί ο server και στη συνέχεια ο client. Τα αποτελέσματα είναι τα εξής :

```
C:\Users\georg>python "D:\PROTECTION\novi\server_phase_1.py"  
[+] Server ξεκίνησε και περιμένει συνδέσεις...  
[+] Σύνδεση από: ('192.168.1.229', 53732)  
[+] Εισάγετε μήνυμα προς τον client: [ ] Λήψη μηνύματος από client: geia  
[+] Λήψη μηνύματος από client: ti kaneis  
geia sou kala eimai  
[+] Εισάγετε μήνυμα προς τον client: kano dokimes gia na doume an stelνονται sosta ta paketa  
[+] Εισάγετε μήνυμα προς τον client: esy  
[+] Εισάγετε μήνυμα προς τον client: [ ] Λήψη μηνύματος από client: oraia
```

Στην πρώτη εικόνα βλέπουμε τα αποτελέσματα από την πλευρά του server και στην δεύτερη εικόνα για τον client. Για να δούμε ότι τα πακέτα στάλθηκαν σωστά χρησιμοποιούμε το εργαλείο wireshark και εστιάζουμε στην συνδεση των δυο IP με την χρήση του φίλτρου `ip.addr == 192.168.1.84 && ip.addr == 192.168.1.229` . Τα αποτελέσματα είναι τα εξής :

1474..	571.084027	192.168.1.229	192.168.1.84	TCP	54	53732 → 12345	[ACK] Seq=1 Ack=1 Win=131328 Len=0
1476..	573.227218	192.168.1.229	192.168.1.84	TCP	58	53732 → 12345	[PSH, ACK] Seq=1 Ack=1 Win=131328 Len=4
1476..	573.281794	192.168.1.84	192.168.1.229	TCP	54	12345 → 53732	[ACK] Seq=1 Ack=5 Win=131328 Len=0
1484..	579.997341	192.168.1.229	192.168.1.84	TCP	63	53732 → 12345	[PSH, ACK] Seq=5 Ack=1 Win=131328 Len=9
1484..	580.055117	192.168.1.84	192.168.1.229	TCP	54	12345 → 53732	[ACK] Seq=1 Ack=14 Win=131328 Len=0
1490..	585.642896	192.168.1.84	192.168.1.229	TCP	74	12345 → 53732	[PSH, ACK] Seq=1 Ack=14 Win=131328 Len=20
1490..	585.689376	192.168.1.229	192.168.1.84	TCP	54	53732 → 12345	[ACK] Seq=14 Ack=21 Win=131328 Len=0
1503..	598.858139	192.168.1.84	192.168.1.229	TCP	109	12345 → 53732	[PSH, ACK] Seq=21 Ack=14 Win=131328 Len=55
1503..	598.912476	192.168.1.229	192.168.1.84	TCP	54	53732 → 12345	[ACK] Seq=14 Ack=76 Win=131072 Len=0
1504..	599.936189	192.168.1.84	192.168.1.229	TCP	58	12345 → 53732	[PSH, ACK] Seq=76 Ack=14 Win=131328 Len=4
1504..	599.986971	192.168.1.229	192.168.1.84	TCP	54	53732 → 12345	[ACK] Seq=14 Ack=80 Win=131072 Len=0
1516..	609.924348	192.168.1.229	192.168.1.84	TCP	59	53732 → 12345	[PSH, ACK] Seq=14 Ack=80 Win=131072 Len=5
1516..	609.985960	192.168.1.84	192.168.1.229	TCP	54	12345 → 53732	[ACK] Seq=80 Ack=19 Win=131328 Len=0

> Frame 147682: 58 bytes on wire (464 bits), 58 bytes captured (464 bits) on interface \Device\NPF_{D43282...}	0000	28 cd c4 97 ae 03 8c c8 4b 5e a8 37 08 00 45 00	K^7..E..
> Ethernet II, Src: ChongqingFug_5e:a8:37 (8c:c8:4b:5e:a8:37), Dst: ChongqingFug_97:ae:03 (28:cd:c4:97:ae:03)	0010	00 2c e5 07 40 00 00 06 91 3a c0 a8 01 e5 c0 a8	@.....
> Internet Protocol Version 4, Src: 192.168.1.229, Dst: 192.168.1.84	0020	01 54 d1 e4 30 39 7f 18 91 70 5b 25 19 98 50 18	T...09...p[%..P..
> Transmission Control Protocol, Src Port: 53732, Dst Port: 12345, Seq: 1, Ack: 1, Len: 4	0030	02 01 d1 12 00 00 67 65 69 61	ge ia
▼ Data (4 bytes)			
Data: 67656961			
[Length: 4]			

Παρατηρούμε ότι υπάρχει σύνδεση μεταξύ server και client. Τα περισσότερα πακέτα έχουν το flag [ACK], που σημαίνει ότι επιβεβαιώνουν τη λήψη δεδομένων. Υπάρχουν και πακέτα με flag [PSH, ACK], που υποδηλώνουν αποστολή δεδομένων με άμεση παράδοση στον ανώτερο επίπεδο. Οι αριθμοί ακολουθίας (Seq) και επιβεβαίωσης (Ack) αυξάνονται σταδιακά, γεγονός που δείχνει μια συνεχή ροή δεδομένων με μικρό μήκος πακέτων (Len). Οι δύο χρήστες στέλνουν TCP πακέτα μεταξύ τους χωρίς να είναι κρυπτογραφημένα. Αυτό μπορούμε να το δούμε στο κάτω μέρος της εικόνας που εμφανίζεται το περιεχόμενο ενός πακέτου σε μορφή hexadecimal και ASCII . Το μήνυμα ήταν geia και όπως βλέπουμε εμφανίζεται κανονικά.

Φάση 2: Προστασία της επικοινωνίας με TLS/SSL

Σε αυτή την φάση θέλουμε να πάρουμε την δομή της φάσης 1 και να την προσαρμόσουμε έτσι ώστε η επικοινωνία να είναι με την χρήση TLS/SSL . Για να το πετύχουμε αυτό θα πρέπει και οι δυο κωδικες να τρέχουν με το ίδιο πιστοποιητικό (ca.cert.pem) .

Ο κώδικας server_phase_2 θα χρησιμοποιεί τα πιστοποιητικά ca.cert.pem , server.cert.pem με τα αντίστοιχα κλειδιά και ο client τα ca.cert.pem Και client.cert.pem . Και τα δυο πιστοποιητικά client Και server πρέπει να είναι υπογεγραμμένα με την πιστοποίηση του ca.cert.pem.

Για την δημιουργία των πιστοποιητικών χρησιμοποιήθηκε το εργαλείο OPENSSL

Αρχικά δημιουργήσαμε ένα κλειδί για το certification που χρησιμοποιούν και οι δυο χρήστες ca.cert.pem.

openssl genrsa -out ca.key.pem 2048

Δημιουργία self-signed πιστοποιητικού για την CA .Σε αυτο τον κώδικα μας ζητήθηκε να βάλουμε και κάποια στοιχεία όπως πως να λεγεται το ca μας και αρκετά άλλα που δείχνουμε στην συνέχεια.

openssl req -x509 -new -nodes -key ca.key.pem -sha256 -days 365 -out ca.cert.pem

```
D:\PROTECTION\New folder\CA\certs>openssl x509 -in ca.cert.pem -noout -text
Certificate:
    Data:
        Version: 3 (0x2)
        Serial Number:
            47:bc:26:b5:79:6a:f3:66:2d:9f:ed:47:3d:91:a0:ed:01:b9:c8:c4
        Signature Algorithm: sha256WithRSAEncryption
        Issuer: C=GR, ST=XANTHI, L=XANTH, O=DUTH, OU=EE, CN=MYCA, emailAddress=georgetoumaras@gmail.com
        Validity
            Not Before: Jan 13 16:37:14 2025 GMT
            Not After : Jan 11 16:37:14 2035 GMT
        Subject: C=GR, ST=XANTHI, L=XANTH, O=DUTH, OU=EE, CN=MYCA, emailAddress=georgetoumaras@gmail.com
        Subject Public Key Info:
            Public Key Algorithm: rsaEncryption
            Public-Key: (4096 bit)
            Modulus:
```

Certificates για τον Server

Ιδιωτικό κλειδί για τον server

openssl genrsa -out server.key.pem 2048

Δημιουργία CSR (Certificate Signing Request) για τον server. Και εδώ μας ζητήθηκε να συμπληρώσουμε κάποια στοιχεία.

openssl req -new -key server.key.pem -out server.csr.pem

```
D:\PROTECTION\New folder\CA>openssl x509 -in server.cert.pem -noout -text
Certificate:
    Data:
        Version: 3 (0x2)
        Serial Number:
            44:ed:60:5d:41:93:a4:58:b6:48:32:b2:d9:bf:c5:24:80:3e:df:05
        Signature Algorithm: sha256WithRSAEncryption
        Issuer: C=GR, ST=XANTHI, L=XANTH, O=DUTH, OU=EE, CN=MYCA, emailAddress=georgetoumaras@gmail.com
        Validity
            Not Before: Jan 13 16:38:09 2025 GMT
            Not After : Jan 23 16:38:09 2026 GMT
        Subject: C=GR, ST=GREECE, L=XANTHI, O=DUTH, OU=EE, CN=GEORGE TOUMARAS, emailAddress=geortoum1@ee.duth.gr
        Subject Public Key Info:
            Public Key Algorithm: rsaEncryption
            Public-Key: (4096 bit)
            Modulus:
```

Υπογραφή του CSR του server με την CA:

**openssl x509 -req -in server.csr.pem -CA ca.cert.pem -CAkey ca.key.pem
-CAcreateserial -out server.cert.pem -days 365 -sha256**

Certificates για τον client

Ομοίως όπως στο server εκτελούμε τις εντολές αλλά αυτή τη φορά από την μεριά του client.

openssl genrsa -out client.key.pem 2048

openssl req -new -key client.key.pem -out client.csr.pem

```
D:\PROTECTION\New folder\CA>openssl x509 -in client.cert.pem -noout -text
Certificate:
    Data:
        Version: 3 (0x2)
        Serial Number:
            44:ed:60:5d:41:93:a4:58:b6:48:32:b2:d9:bf:c5:24:80:3e:df:06
        Signature Algorithm: sha256WithRSAEncryption
        Issuer: C=GR, ST=XANTHI, L=XANTH, O=DUTH, OU=EE, CN=MYCA, emailAddress=georgetoumaras@gmail.com
        Validity
            Not Before: Jan 13 16:38:52 2025 GMT
            Not After : Jan 23 16:38:52 2026 GMT
        Subject: C=GR, ST=GREECE, L=XANTHI, O=DUTH, OU=EE, CN=giorgos agakechagias, emailAddress=georgetoumaras@gmail.com
        Subject Public Key Info:
            Public Key Algorithm: rsaEncryption
            Public-Key: (2048 bit)
            Modulus:
```

```
openssl x509 -req -in client.csr.pem -CA ca.cert.pem -CAkey ca.key.pem
-CACreateserial -out client.cert.pem -days 365 -sha256
```

Τελική Δομή Αρχείων

Μετά την εκτέλεση των ακόλουθων εντολών στο cmd των windows θα πρέπει να έχουν δημιουργηθεί τα εξής αρχεία:

CA:

- ca.key.pem: Ιδιωτικό κλειδί της CA.
- ca.cert.pem: Self-signed πιστοποιητικό της CA.

Server:

- server.key.pem: Ιδιωτικό κλειδί του server.
- server.csr.pem: Certificate Signing Request του server.
- server.cert.pem: Υπογεγραμμένο πιστοποιητικό του server.

Client:

- client.key.pem: Ιδιωτικό κλειδί του client.
- client.csr.pem: Certificate Signing Request του client.
- client.cert.pem: Υπογεγραμμένο πιστοποιητικό του client.
-

Για να επαληθεύσουμε ότι τα πιστοποιητικά μας είναι verified μπορούμε να τρέξουμε τον κώδικα **verify -CAFILE ca.cert.pem server.cert.pem**. Ο σκοπός της εντολής είναι να διασφαλίσει ότι το πιστοποιητικό του server είναι έγκυρο και εκδόθηκε από την αξιόπιστη αρχή πιστοποίησης (CA). Αντίστοιχα και για τον client. Η απάντηση και στα δύο είναι Ok οπότε και τα δύο certs είναι επιβεβαιωμένα με την CA.

```
D:\PROTECTION\New folder\CA>openssl verify -CAfile ca.cert.pem server.cert.pem
server.cert.pem: OK
```

```
D:\PROTECTION\New folder\CA>openssl verify -CAfile ca.cert.pem client.cert.pem
client.cert.pem: OK
```

Κώδικας server_phase_2

Ο νέος αυτός κώδικας προσθέτει ασφάλεια μέσω SSL/TLS. Αυτό σημαίνει ότι η επικοινωνία μεταξύ client και server είναι κρυπτογραφημένη, προσφέροντας ασφάλεια και έναντι κλοπής και παραποίησης.

Αρχικά εισάγουμε η βιβλιοθήκη ssl, η οποία χρησιμοποιείται για την υλοποίηση κρυπτογραφημένων συνδέσεων

```
import ssl
```

Εδώ βάζουμε τα directories από τα πιστοποιητικά που δημιουργήσαμε εμείς . Συγκεκριμένα για το private_key και certificate βάζουμε τα αντίστοιχα του server και για το ca_cert χρησιμοποιούμε το πιστοποιητικό που μοιράζονται και οι δύο χρήστες (CA_CERT).

```
CERTIFICATE = "D:/PROTECTION/New folder/CA/server.cert.pem"
PRIVATE_KEY = "D:/PROTECTION/New folder/CA/server.key.pem"
CA_CERT = "D:/PROTECTION/New folder/CA/ca.cert.pem"
```

Δημιουργούμε ένα περιβάλλον για αυθεντικοποίηση του server , φορτώνουμε το κλειδί και το πιστοποιητικό του και ορίζουμε το πιστοποιητικό της ca για την επαλήθευση του server . Επίσης όταν ο client είναι υποχρεωμένος να παρουσιάσει έγκυρο πιστοποιητικό πιστοποιημένο από το CA.

```
context = ssl.create_default_context(ssl.Purpose.CLIENT_AUTH)
context.load_cert_chain(certfile=CERTIFICATE, keyfile=PRIVATE_KEY)
context.load_verify_locations(CA_CERT)
context.verify_mode = ssl.CERT_REQUIRED # Υποχρεωτική πιστοποίηση του client
```

Εδώ μετατρέπουμε τα TCP sockets σε ασφαλές socket με SSL/TLS και δηλώνουμε ότι το socket λειτουργεί ως server

```
ssl_client_socket = context.wrap_socket(client_socket, server_side=True)
```

Αμα τα πιστοποιητικά που χρησιμοποιούμε δεν είναι έγκυρα βγαζει αυτο το σφάλμα. Αν παρουσιασει σφάλμα, η σύνδεση κλείνει με ασφάλεια.

```
except ssl.SSLError as e:
    print(f"❌ Σφάλμα SSL: {e}")
```

Κώδικας client_phase_2.

Όπως και στον server_phase_2 στην αρχή εισάγουμε τη βιβλιοθήκη ssl και τα αντίστοιχα πιστοποιητικά του client

```
CLIENT_CERT = "D:\stelno\CA\client.cert.pem"  
CLIENT_KEY = "D:\stelno\CA\client.key.pem"  
CA_CERT = "D:\stelno\CA\ca.cert.pem"
```

Το κανονικό socket κρυπτογραφείται με SSL/TLS. Το GEORGE TOUMARAS χρησιμοποιείται για την επαλήθευση του server (π.χ., το κοινό όνομα στο πιστοποιητικό πρέπει να ταιριάζει)

```
ssl_client_socket = context.wrap_socket(client_socket, server_hostname="GEORGE TOUMARAS")
```

Σημαντικό κομμάτι του κώδικα είναι να αντιστοιχίσουμε την IP του server με τον κώδικα μας με αποτέλεσμα ο client να συνδέεται με ασφάλεια στο port 12345

```
ssl_client_socket.connect(("192.168.1.84", 12345))
```

Τι Προσφέρει η SSL/TLS Σύνδεση

Κρυπτογράφηση Δεδομένων: Τα δεδομένα που αποστέλλονται και λαμβάνονται είναι κρυπτογραφημένα, προστατεύοντας τα από υποκλοπή.

Αuthεντικοποίηση Server: Ο client επαληθεύει ότι συνδέεται με τον σωστό server μέσω του πιστοποιητικού.

Ασφάλεια Εμπιστευτικότητας: Ακόμα και αν κάποιος υποκλέψει τα δεδομένα, δεν μπορεί να τα διαβάσει.

Σύνδεση μεταξύ server_phase_2 και client_phase_2

Αφότου κάναμε τις τροποποιήσεις στον κώδικα μας μπορούμε να τρέξουμε τους κώδικες μας και να δούμε τα αποτελέσματα που θα μας βγάλει το εργαλείο wireshark με την χρήση του φίλτρου `ip.addr == 192.168.1.84 && ip.addr == 192.168.1.229`.

```
C:\Users\georg>cd /d D:\PROTECTION\novi\kodikes  
  
D:\PROTECTION\novi\kodikes>python server_phase_2.py  
[+] TLS Server ξεκίνησε και περιμένει συνδέσεις...  
[+] Σύνδεση από: ('192.168.1.229', 1600)  
[+] Εισάγετε μήνυμα προς τον client (ή 'exit' για έξοδο): geia s  
[+] Εισάγετε μήνυμα προς τον client (ή 'exit' για έξοδο): [M] Μήνυμα από client: geia  
[+] Μήνυμα από client: geia  
ola kala  
[+] Εισάγετε μήνυμα προς τον client (ή 'exit' για έξοδο): [M] Μήνυμα από client: kaigw  
xairomai file [M] Μήνυμα από client: pws pei h zwh  
[+] Εισάγετε μήνυμα προς τον client (ή 'exit' για έξοδο): ola kala  
[+] Εισάγετε μήνυμα προς τον client (ή 'exit' για έξοδο): [M]
```



```
C:\Users\giorg>python "D:\ΑΣΦΑΛΕΙΑ_ΣΥΣΤ\client_phase2.py"
[?] Συνδέθηκε με επιτυχία στον Server μέσω TLS!
[?] Εισάγετε μήνυμα προς τον Server (ή 'exit' για έξοδο): geia [?] Μήνυμα από server: geia s
[?] Εισάγετε μήνυμα προς τον Server (ή 'exit' για έξοδο): geia
[?] Εισάγετε μήνυμα προς τον Server (ή 'exit' για έξοδο): [?] Μήνυμα από server: ola kala
kaigw
[?] Εισάγετε μήνυμα προς τον Server (ή 'exit' για έξοδο): pws rei h zwh
[?] Εισάγετε μήνυμα προς τον Server (ή 'exit' για έξοδο): [?] Μήνυμα από server: ola kala
ena duo ena duo
[?] Εισάγετε μήνυμα προς τον Server (ή 'exit' για έξοδο):
```

ip.addr == 192.168.1.84 && ip.addr == 192.168.1.229						
No.	Time	Source	Destination	Protocol	Length	Info
280	7.0748...	192.168...	192.168.1.229	TLSv1.2	85	Application Data
281	7.1301...	192.168...	192.168.1.84	TCP	54	1600 → 12345 [ACK] Seq=1 Ack=32 Win=513 Len=0
889	11.068...	192.168...	192.168.1.84	TLSv1.2	81	Application Data
895	11.121...	192.168...	192.168.1.229	TCP	54	12345 → 1600 [ACK] Seq=32 Ack=28 Win=512 Len=0
10...	14.347...	192.168...	192.168.1.84	TLSv1.2	89	Application Data
10...	14.394...	192.168...	192.168.1.229	TCP	54	12345 → 1600 [ACK] Seq=32 Ack=63 Win=512 Len=0
47...	32.381...	192.168...	192.168.1.229	TLSv1.2	85	Application Data
47...	32.430...	192.168...	192.168.1.84	TCP	54	1600 → 12345 [ACK] Seq=63 Ack=63 Win=512 Len=0
49...	39.089...	192.168...	192.168.1.84	TLSv1.2	91	Application Data
50...	39.144...	192.168...	192.168.1.229	TCP	54	12345 → 1600 [ACK] Seq=63 Ack=100 Win=512 Len=0

Frame 1039: 54 bytes on wire (432 bits), 54 bytes captured (432 bits) on interface \Device\NPF_{3B3DFF86-0000-8c c8 4b 5e a8 37 28 cd c4 97 ae 03 00 00 45 00} ...K^ 7(. E	0000	8c c8 4b 5e a8 37 28 cd c4 97 ae 03 00 00 45 00
Ethernet II, Src: ChongqingFug_97:ae:03 (28:cd:c4:97:ae:03), Dst: ChongqingFug_5e:a8:37 (8c:c8:4b:5e:a8:37) ...h0 T	0010	00 28 e0 62 40 00 80 06 95 e3 c0 a8 01 54 c0 a8
Internet Protocol Version 4, Src: 192.168.1.84, Dst: 192.168.1.229	0020	01 e5 30 39 06 40 d9 05 bc 10 de ea cc 2e 50 10
Transmission Control Protocol, Src Port: 12345, Dst Port: 1600, Seq: 32, Ack: 63, Len: 0	0030	02 00 b2 a1 00 00

Όπως μπορούμε και να διακρίνουμε και στο wireshark όντως τα πακέτα στέλνονται επιτυχώς με TCP και TLS απο τον ενα ip 192.168.1.84(server) στο 192.168.1.229(client) .Υπάρχει συνεχής ανταλλαγή πακέτων μεταξύ των δύο διευθύνσεων IP, κυρίως μέσω του TLSv1.2, που σημαίνει ότι η επικοινωνία είναι κρυπτογραφημένη.Τα ACK πακέτα δείχνουν σταθερή επιβεβαίωση λήψης δεδομένων.Ο ακριβής τύπος των δεδομένων δεν μπορεί να προσδιοριστεί λόγω της κρυπτογράφησης.Το TLSv1.2 είναι μια ασφαλής έκδοση του πρωτοκόλλου SSL/TLS που χρησιμοποιείται για ασφαλή επικοινωνία.

Το hex dump εμφανίζει τα δεδομένα του πακέτου, με δυαδική αναπαράσταση και ASCII απεικόνιση. Τα δεδομένα είναι κρυπτογραφημένα (TLSv1.2), άρα το περιεχόμενο δεν είναι άμεσα αναγνώσιμο.

Φάση 3: Προστασία της επικοινωνίας με IPsec

Μια εναλλακτική μέθοδος για την προστασία του καναλιού επικοινωνίας είναι η χρήση IPsec μηχανισμού. Εδώ θα χρησιμοποιήσουμε τον κώδικα που υλοποιήσαμε στην φαση 1 και θα χρησιμοποιήσουμε το IPsec ώστε να γίνει η ασφάλιση του δικτύου. Το IPsec (Internet Protocol Security) είναι ένα πρωτόκολλο που χρησιμοποιείται για την ασφάλεια της επικοινωνίας στο διαδίκτυο μέσω του πρωτοκόλλου IP. Στόχος είναι να δημιουργήσουμε ενα IPsec το οποίο θα χρησιμοποιεί τα πιστοποιητικά που φτιάξαμε στην φάση 2. Καθώς δυσκολευτήκαμε με την τροποποίηση του IPsec ώστε να χρησιμοποιεί τα δικά μας

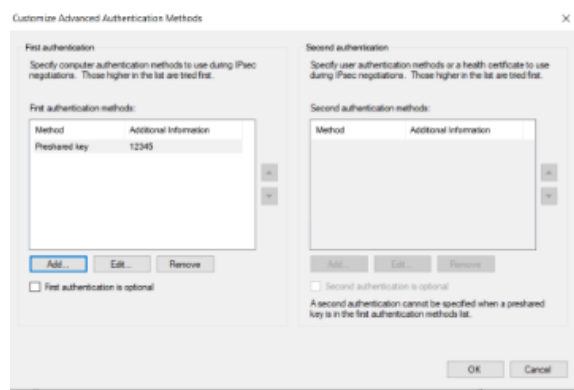
πιστοποιητικά στο λειτουργικό σύστημα windows, χρησιμοποιήσαμε την μέθοδο του pre shared key. Έπειτα χρησιμοποιήσαμε το λειτουργικό σύστημα Linux που έχει καλύτερη διαχείριση των IPsec ρυθμίσεων για να βγάλουμε εις πέρας το ζητούμενο της άσκησης .

ΜΕΘΟΔΟΣ WINDOWS(PRE SHARED KEY)

Με αυτή την μέθοδο θα διαμορφώσουμε έτσι το ipsec ώστε να γίνεται αλληλεπίδραση μεταξύ παραλήπτη και αποστολέα με την χρήση ενός κλειδιού (preshared key) .Όταν χρησιμοποιείται με **preshared key (PSK)**, η ασφάλεια επιτυγχάνεται μέσω της κοινής χρήσης ενός προκαθορισμένου μυστικού κλειδιού μεταξύ των δύο άκρων της σύνδεσης.

Τα βήματα για να δημιουργήσουμε το IPsec στα windows με την χρήση pre shared key είναι τα εξής :

1. Ανοίγουμε το windows defender firewall with advanced security
2. Πατάμε δεξί κλικ στο connection security rules και επιλέγουμε το new rule
3. Σε αυτο το βήμα κανουμε custom το IPsec μας
4. Εισάγουμε τα IP του client και του server. Από την πλευρά του server βάζουμε σαν endpoint 1 του server και 2 του client και αντιστροφα ενεργούμε για τον client .
5. Στο πεδίο when do you want authentication to occur επιλέγουμε το require authentication for inbound and outbound connection η οποία επιτρέπει μόνο τις συνδέσεις μεταξύ εισερχομένων και εξερχομένων που χρησιμοποιούν το ίδιο κλειδί.
6. Στο authentication method πατάμε advanced και custom και βάζουμε το pre shared key που θέλουμε να χρησιμοποιήσουμε.



Αφού κάνει και ο client και ο server την ίδια διαδικασία κάνουμε enable το νέο rule που μόλις προσαρμόσαμε και τρέχουμε τα προγράμματα

Name	Enabled	Local IP Address	Remote IP Address	Action	Authentication Method	Authentication Level	Authentication Type	Protocol
IPsec	Yes	192.168.1.229	192.168.1.84	Require inbound and ...	Custom	Any	Any	TCP

```

Microsoft Windows [Version 10.0.19045.5247]
(c) Microsoft Corporation. All rights reserved.

C:\Users\georg>python "D:\PROTECTION\novi\kodikes\server_phase_1.py"
[?] Server ξεκίνησε και περιμένει συνδέσεις...
[?] Συνδεση από: ('192.168.1.229', 25792)
[?] Εισάγετε μήνυμα προς τον client: [?] Λήψη μηνύματος από client: geia
[?] Λήψη μηνύματος από client: ena duo ena duo
[?] Λήψη μηνύματος από client: akouei akouei
geia sou ti kaneis pos eisia
[?] Εισάγετε μήνυμα προς τον client: ola kala
[?] Εισάγετε μήνυμα προς τον client: [?] Λήψη μηνύματος από client: ena duo

```

```

C:\Users\giorg>python "D:\ΑΣΦΑΛΕΙΑ_ΣΥΣΤ\client_phase1.py"
[?] Συνδέθηκε με επιτυχία στον Server!
[?] Εισάγετε μήνυμα προς τον Server (ή 'exit' για έξοδο): geia
[?] Εισάγετε μήνυμα προς τον Server (ή 'exit' για έξοδο): ena duo ena duo
[?] Εισάγετε μήνυμα προς τον Server (ή 'exit' για έξοδο): akouei akouei
[?] Εισάγετε μήνυμα προς τον Server (ή 'exit' για έξοδο): [?] Απάντηση από Server: geia sou ti kaneis pos eisia
[?] Απάντηση από Server: ola kala
ena duo
[?] Εισάγετε μήνυμα προς τον Server (ή 'exit' για έξοδο):

```

Χρησιμοποιώντας το φίλτρο `ip.addr == 192.168.1.229 && ip.addr == 192.168.1.84` στο Wireshark βλέπουμε τα αντίστοιχα αποτελέσματα. Σε αυτή την μέθοδο στέλνονται κανονικά ESP πακέτα αλλά δεν χρησιμοποιούμε τα `certification` από το δεύτερο μέρος καθώς το λειτουργικό σύστημα που τρέξαμε τα προγράμματα μας Windows 10 Home Edition δεν είχε τόσο προσαρμοστικότητα όσο αφορά στην δημιουργία του IPsec με αποτέλεσμα τα μηνύματα να μην κρυπτογραφούνται σε μορφή ASCII. Όπως μπορούμε να δούμε σε ένα πακέτο που επιλέξαμε στην μορφή ASCII μπορούμε να δούμε το μήνυμα που στάλθηκε `geia`.

ip.addr == 192.168.1.229 && ip.addr == 192.168.1.84						
No.	Time	Source	Destination	Protocol	Length	Info
12	8.251940	192.168.1.229	192.168.1.84	ESP	82	ESP (SPI=0xb04183fe)
13	8.307859	192.168.1.84	192.168.1.229	ESP	78	ESP (SPI=0xff5244e8)
16	10.880324	192.168.1.229	192.168.1.84	ESP	94	ESP (SPI=0xb04183fe)
17	10.932274	192.168.1.84	192.168.1.229	ESP	78	ESP (SPI=0xff5244e8)
61	25.994952	192.168.1.229	192.168.1.84	ESP	90	ESP (SPI=0xb04183fe)
62	26.051916	192.168.1.84	192.168.1.229	ESP	78	ESP (SPI=0xff5244e8)
215	50.950151	192.168.1.84	192.168.1.229	ESP	106	ESP (SPI=0xff5244e8)
216	50.997871	192.168.1.229	192.168.1.84	ESP	78	ESP (SPI=0xb04183fe)
217	52.434293	192.168.1.84	192.168.1.229	ESP	86	ESP (SPI=0xff5244e8)
218	52.476948	192.168.1.229	192.168.1.84	ESP	78	ESP (SPI=0xb04183fe)
244	56.636636	192.168.1.229	192.168.1.84	ESP	86	ESP (SPI=0xb04183fe)
245	56.941046	192.168.1.229	192.168.1.84	ESP	86	ESP (SPI=0xb04183fe)
246	57.108568	192.168.1.84	192.168.1.229	ESP	90	ESP (SPI=0xff5244e8)

> Frame 12: 82 bytes on wire (656 bits), 82 bytes captured (656 bits) on interface \Device\NPF_{D43282E8-...}		0000	28 cd c4 97 ae 03 8c c8 4b 5e a8 37 08 00 45 00	(.....) K^7...E:
> Ethernet II, Src: ChongqingFug_5e:a8:37 (8c:c8:4b:5e:a8:37), Dst: ChongqingFug_97:ae:03 (28:cd:c4:97:ae:03)		0010	00 44 a1 2e 40 00 80 32 d4 cf c0 a8 01 e5 c0 a8	.D...@-2
> Internet Protocol Version 4, Src: 192.168.1.229, Dst: 192.168.1.84		0020	01 54 b0 41 83 fe 00 00 00 03 64 c0 30 39 e3 68	.T.A.....d:09:h
> Encapsulating Security Payload		0030	72 76 5d 8e 76 66 50 18 02 03 99 a7 00 00 67 65	rvJ-vfP.....ge
		0040	69 61 01 02 02 06 59 ca 59 82 9f be 33 d0 08 59	la....Y...3..Y
		0050	3a b8	:

ΜΕΘΟΔΟΣ LINUX

Για την πραγματοποίηση της IPsec χρειάστηκε να κατεβάσουμε το εργαλείο strongswan απο το τερματικό του linux χρησιμοποιώντας τον κώδικα **sudo apt install strongswan**. Έπειτα το μόνο που έλειπε ήταν να φροντίσουμε τα πιστοποιητικά που χρειάστηκαν στους καταλλήλους φακέλους και να τρέξουμε κάποιους κώδικες . Η διαδικασία ήταν η εξής:

Για να περάσουμε τα απαραίτητα certification που χρειαστήκαμε έπρεπε να τα βάλουμε σε σωστούς φακέλους οπότε δημιουργήσαμε τους φακέλους που χρειαζόμασταν

```
sudo mkdir -p /etc/ipsec.d/certs
sudo mkdir -p /etc/ipsec.d/private
sudo mkdir -p /etc/ipsec.d/cacerts
```

Έπειτα αντιγράψαμε τα αρχεία αυτά στους αντίστοιχους φακέλους:

```
sudo cp server.cert.pem /etc/ipsec.d/certs/
sudo cp server.key.pem /etc/ipsec.d/private/
sudo cp ca.cert.pem /etc/ipsec.d/cacerts/
```

Για να μην υπάρχουν κάποια λάθη κατά την εκτέλεση αλλάξαμε τα δικαιώματα εγγραφής για τους φακέλους καθώς και για τα αρχεία:

```
sudo chmod -R 700 /etc/ipsec.d/private
sudo chmod -R 755 /etc/ipsec.d/certs
sudo chmod -R 755 /etc/ipsec.d/cacerts
```

```
sudo chmod 644 /etc/ipsec.d/certs/server.cert.pem
sudo chmod 600 /etc/ipsec.d/private/server.key.pem
sudo chmod 644 /etc/ipsec.d/cacerts/ca.cert.pem
```

Συνοπτικά η εντολή **chmod 700** ορίζει τον κάτοχο του αρχείου να έχει πλήρη δικαιώματα ανάγνωσης, εγγραφής και εκτέλεσης , το 755 σημαίνει ότι ο κάτοχος έχει πλήρη δικαιώματα, η ομάδα έχει δικαιώματα ανάγνωσης και εκτέλεσης, ενώ οι άλλοι χρήστες έχουν μόνο δικαίωμα ανάγνωσης. Το 644 σημαίνει ότι ο κάτοχος έχει δικαιώματα ανάγνωσης και εγγραφής, η ομάδα και οι άλλοι χρήστες έχουν μόνο δικαίωμα ανάγνωσης και το 600 που μόνο ο κάτοχος έχει δικαίωμα ανάγνωσης και εγγραφής

Επεξεργασία ipsec.conf με την χρήση του κώδικα sudo nano /etc/ipsec.conf

```
config setup
    charondebug="ike 1, knl 1, cfg 0"
    uniqueids=no

conn server-to-client
    left=192.168.1.193
    leftcert=/etc/ipsec.d/certs/server.cert.pem
    leftid=@server
    leftsubnet=0.0.0.0/0
    right=192.168.1.159
    rightcert=/etc/ipsec.d/certs/client.cert.pem
    rightid=@client
    auto=add
    esp=aes256-sha256-modp2048
```

Το config setup περιέχει γενικές ρυθμίσεις για το StrongSwan όπως το επίπεδο καταγραφής (IKE) ,ο πυρήνας (kernel) και η διαμόρφωση (configuration).

Το **conn server-to-client** είναι μια σύνδεση VPN που χρησιμοποιούμε μεταξύ του server και του client. Στο πεδίο left βάζουμε το IP του server, στην περίπτωση του server 192.168.1.193, στο leftcert το πιστοποιητικό του server leftid=@server το αναγνωριστικό του server (που θα μας χρειαστεί για τα κλειδιά σε επόμενο βήμα) και το leftsubnet=0.0.0.0/0 που δημιουργεί το υποδίκτυο που θα δρομολογηθεί μέσω του server . Από την πλευρά right βάλουμε τα στοιχεία του client και το πιστοποιητικό του.

Με την χρήση του **esp=aes256-sha256-modp2048** ορίζουμε την κρυπτογράφηση που θέλουμε για την προστασία των δεδομένων μας.

Πιο συγκεκριμένα :

aes256: Αλγόριθμος κρυπτογράφησης AES με μήκος κλειδιού 256 bit.

sha256: Αλγόριθμος σύνοψης για πιστοποίηση μηνυμάτων.

modp2048: Αλγόριθμος Diffie-Hellman για ασφαλή ανταλλαγή κλειδιών, με μήκος κλειδιού 2048 bit.

Ρύθμιση χρηστών με την χρήση του sudo nano /etc/ipsec.secrets

```
@server : RSA /etc/ipsec.d/private/server.key.pem
@client  : RSA /etc/ipsec.d/private/client.key.pem
```

Εδώ βάζουμε τα κλειδιά του server και του client. Χρησιμοποιείται για τη διαδικασία πιστοποίησης μέσω του πρωτοκόλλου RSA.

Κάναμε restart το IPsec ώστε να μπορέσουμε να τρέξουμε το διαμορφωποιημένο IPsec.

```
linux@linux-VirtualBox:~$ sudo ipsec restart
[sudo] password for linux:
Stopping strongSwan IPsec...
Starting strongSwan 5.9.13 IPsec [starter]...
```

Αντίστοιχα κάνουμε τα ίδια βήματα και για τον client. Αυτή τη φορά στο `ipsec.conf` δημιουργούμε ένα νη client-to-server και βάζουμε τα αντίστοιχα. Όπως μπορούμε να διακρίνουμε αυτή τη φορά βάλαμε πρώτα το πιστοποιητικό του client.

```
config setup
    charondebug="ike 1, knl 1, cfg 0"
    uniqueids=no

conn client-to-server
    left=192.168.1.159
    leftcert=/etc/ipsec.d/certs/client.cert.pem
    leftid=@client

    right=192.168.1.193
    rightcert=/etc/ipsec.d/certs/server.cert.pem
    rightid=@server

    auto=add
    esp=aes256-sha256-modp2048
```

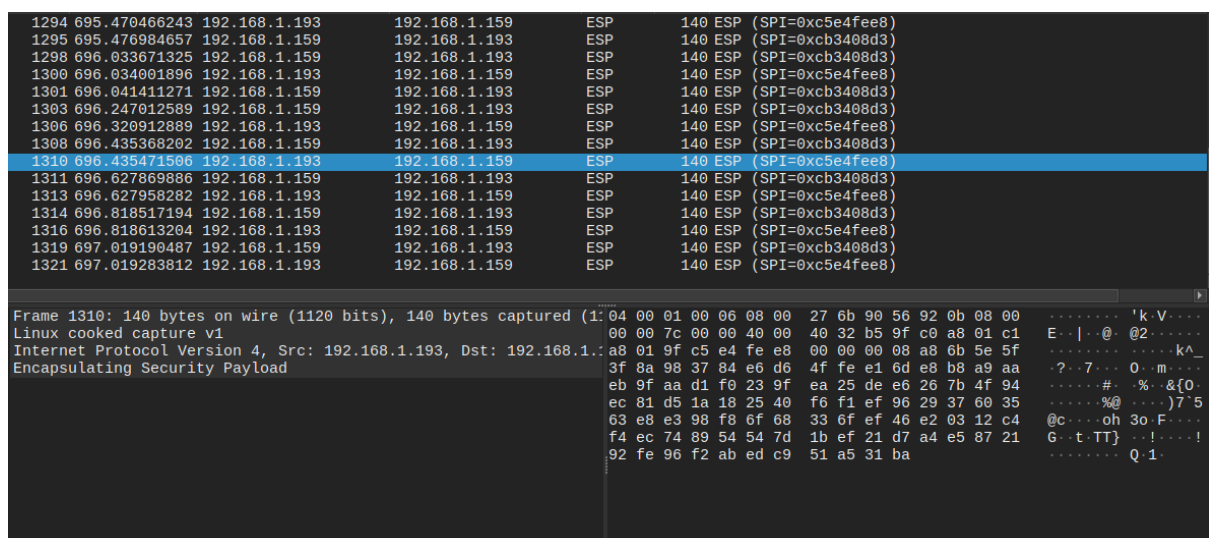
Το μόνο που έμενε ήταν να τρέξουμε το IPsec μας με την εντολή **sudo ipsec up server-to-client** και η σύνδεση μας είχε επιτευχθεί.

Τροποποιώντας το `client_phase_1` ώστε πλέον να έχει την IP του Linux αυτή τη φορά 192.168.1.159 τρέξαμε τα προγράμματα και η σύνδεση έγινε με επιτυχία.

```
linux@linux-VirtualBox: ~/Downloads
linux@linux-VirtualBox:~$ cd /home/linux/Downloads
linux@linux-VirtualBox:~/Downloads$ python3 server_phase_1.py
Server ξεκίνησε και περιμένει συνδέσεις...
Σύνδεση από: ('192.168.1.159', 44354)
Εισάγετε μήνυμα προς τον client: ela ti kanei
Λήψη μηνύματος από client: ti kaneis
ala eimai
Εισάγετε μήνυμα προς τον client: dokimazo an egine sundesi
Εισάγετε μήνυμα προς τον client:
```

```
linux@linux: ~/Downloads/chat
linux@linux:~$ cd /home/linux/Downloads/chat
linux@linux:~/Downloads/chat$ python3 client_phase_1.py
🔒 Συνδέθηκε με επιτυχία στον Server!
💬 Εισάγετε μήνυμα προς τον Server (ή 'exit' για έξοδο): geia
💬 Εισάγετε μήνυμα προς τον Server (ή 'exit' για έξοδο): ti kaneis
💬 Εισάγετε μήνυμα προς τον Server (ή 'exit' για έξοδο): 📩 Ανάντηση από Server:
kala eimai
📩 Ανάντηση από Server: dokimazo an egine sundesi
```

Χρησιμοποιήσαμε και αυτή τη φορά το wireshark για να δούμε αν έχουμε esp πακέτα γράφοντας στα φίλτρα ESP. Βλέπουμε ότι όντως τα πακέτα μεταξύ παραλήπτη και αποστολέα στέλνονται με ESP πρωτόκολλο ένα βασικό πρωτόκολλο του IPsec . Όλα τα δεδομένα είναι κρυπτογραφημένα και το ίδιο SPI αντιστοιχεί στα πακέτα, κάτι που δείχνει σωστή αντιστοίχιση με τις παραμέτρους ασφάλειας.



Φάση 4: Παροχή ανωνυμοποίησης με την χρήση Tor

Το δίκτυο Tor (The Onion Router) είναι ένα σύστημα ανωνυμίας που επιτρέπει στους χρήστες να περιηγούνται στο διαδίκτυο με αυξημένο επίπεδο ανωνυμίας και προστασίας της ιδιωτικότητας. Χρησιμοποιεί έναν κατακεντρωμένο τρόπο δρομολόγησης δεδομένων μέσω πολλών "στρωμάτων" κρυπτογράφησης, κάτι που καθιστά δύσκολη την παρακολούθηση των δραστηριοτήτων του χρήστη ή την αποκάλυψη της τοποθεσίας του.

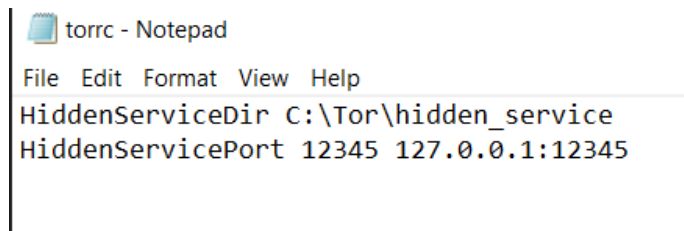
Στην περίπτωση μας θέλαμε στον κώδικα του μέρους πρώτου να περνάει από ένα σύστημα ανωνυμίας ώστε να μην μπορούμε να διακρίνουμε την IP από τα πακέτα που στέλνει ο παραλήπτης και ο αποστολέας.

Αρχικά έπρεπε να κατεβάσουμε το TOR δίκτυο και έπειτα να το διαμορφώσουμε ώστε να δημιουργήσουμε ένα καινούργιο socket.

Δημιουργήσαμε ένα φάκελο για τα hidden services με τον κώδικα

```
mkdir C:\Tor\hidden_service
```

Στον φάκελο που εγκαταστήσαμε το TOR δημιουργήσαμε ένα αρχείο με όνομα torrc . Ουσιαστικά καθορίζουμε τον φάκελο του υπολογιστή που θα αποθηκεύονται τα αρχεία που σχετίζονται με την κρυφή υπηρεσία και καθορίσαμε την διαμόρφωση της κρυφής υπηρεσίας.



Ανοίξαμε το τερματικό για να τρέξουμε το **tor.exe -f torrc** ο οποίος κώδικας μας δημιουργεί ένα αρχείο hostname. Το ανοίγουμε και αυτο παίρνουμε την διεύθυνση .onion και την προωθούμε στον client. Το .onion είναι ένας ειδικός τύπος διεύθυνσης ιστότοπου που δεν μπορεί να βρεθεί με τις συνήθεις μηχανές αναζήτησης και προσφέρει ένα υψηλό επίπεδο ανωνυμίας.

Από την πλευρά του client εκτελούμε τα ίδια βήματα απλώς θα πρέπει να εγκαταστήσουμε και το socks ώστε να τον εντάξουμε στον κώδικα του client . Αυτό γίνεται με την εντολή **pip install requests[socks]**. Το SOCKS είναι ένα πρωτόκολλο διαδικτύου που επιτρέπει σε ένα πρόγραμμα να συνδεθεί σε μια υπηρεσία δικτύου μέσω ενός διακομιστή μεσολάβησης (proxy server).

Μετά μένει να τροποποιήσουμε στους κώδικες τα εξής. Στον κώδικα του server που αντί για 0.0.0.0 που είχαμε βάζουμε το νέο μας σύστημα. Ο server λειτουργεί μέσω Tor ως Hidden Service.

```
server_socket.bind(("127.0.0.1", 12345))
```

Στον κώδικα client βάζουμε την βιβλιοθήκη socks και τροποποιούμε τον κώδικα. Ρυθμίζουμε τον διακομιστή ώστε να ακούει στο 127.0.0.1 και στην θύρα 9050 και ορίζουμε την την διεύθυνση onion του Server.


```

import socks
import socket

#Σύνδεση μέσω Tor (SOCKS5)
socks.set_default_proxy(socks.SOCKS5, "127.0.0.1", 9050)
socket.socket = socks.socksocket

#Διεύθυνση Hidden Service (αντικατάστησε με τη δική σου .onion)
onion_address = "hkd52ovfzvbbndnzxs5cikyosjghsjpdnepptqmcilmjfw22ra2ceqd.onion"

#Δημιουργία του client socket
client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

#Σύνδεση στο Hidden Service μέσω Tor
client_socket.connect((onion_address, 12345))
print(" Συνδέθηκε επιτυχώς στο Hidden Service!")

```

Αφού κάνουμε τις απαραίτητες τροποποιήσεις του κώδικα εκτελούμε την εντολή tor.exe και στους δύο υπολογιστές τρέχουμε τα προγράμματα. Η σύνδεση γίνεται κανονικά και τα αποτελέσματα που μας δείχνει το wireshark με την χρήση του φίλτρου **tcp.port == 9050** είναι αποδεκτό αφού κρύβεται η ip και του αποστολέα καθώς και του παραλήπτη.

```

C:\Users\georg>python "D:\PROTECTION\novi\kodikes\tor\server_phase_1.py"
Tor Hidden Service Server ξεκίνησε και περιμένει συνδέσεις...
Σύνδεση μέσω Tor από: ('127.0.0.1', 22207)
Εισάγετε μήνυμα προς τον client: ti kaneis
Εισάγετε μήνυμα προς τον client: geia
Εισάγετε μήνυμα προς τον client: pos eisia
Εισάγετε μήνυμα προς τον client: ola kala
Εισάγετε μήνυμα προς τον client: xairomai
Εισάγετε μήνυμα προς τον client: [Enter] Λήψη μηνύματος από client: gegeia file
Λήψη μηνύματος από client: ola kal eimai
Λήψη μηνύματος από client: ti eisai ti eimai

```

```

C:\Users\giorg>python "D:\ΑΣΦΑΛΕΙΑ_ΣΥΣΤ\client_phase_tor.py"
[Enter] Συνδέθηκε επιτυχώς στο Hidden Service!
[Enter] Εισάγετε μήνυμα προς τον Server (ή 'exit' για έξοδο): [Enter] Απάντηση από Server: ti kaneis
ge[Enter] Απάντηση από Server: geia
[Enter] Απάντηση από Server: pos eisia
geia f[Enter] Απάντηση από Server: ola kala
file
[Enter] Εισάγετε μήνυμα προς τον Server (ή 'exit' για έξοδο): [Enter] Απάντηση από Server: xairomai
ola kal eimai
[Enter] Εισάγετε μήνυμα προς τον Server (ή 'exit' για έξοδο): ti eisai ti eimai
[Enter] Εισάγετε μήνυμα προς τον Server (ή 'exit' για έξοδο):

```

tcp.port == 9050						
No.	Time	Source	Destination	Protocol	Length	Info
20	2.084283	127.0.0.1	127.0.0.1	TCP	44	26441 → 9050 [ACK] Seq=1 Ack=22 Win=10233 Len=0
21	133.512645	127.0.0.1	127.0.0.1	TCP	69	26441 → 9050 [PSH, ACK] Seq=1 Ack=22 Win=10233 Len=25
22	133.512705	127.0.0.1	127.0.0.1	TCP	44	9050 → 26441 [ACK] Seq=22 Ack=26 Win=10233 Len=0
23	254.974100	127.0.0.1	127.0.0.1	TCP	52	26441 → 9050 [PSH, ACK] Seq=26 Ack=22 Win=10233 Len=8
24	254.974148	127.0.0.1	127.0.0.1	TCP	44	9050 → 26441 [ACK] Seq=22 Ack=34 Win=10233 Len=0
25	256.695287	127.0.0.1	127.0.0.1	TCP	50	26441 → 9050 [PSH, ACK] Seq=34 Ack=22 Win=10233 Len=6
26	256.695341	127.0.0.1	127.0.0.1	TCP	44	9050 → 26441 [ACK] Seq=22 Ack=40 Win=10233 Len=0
27	257.288903	127.0.0.1	127.0.0.1	TCP	46	26441 → 9050 [PSH, ACK] Seq=40 Ack=22 Win=10233 Len=2
28	257.288966	127.0.0.1	127.0.0.1	TCP	44	9050 → 26441 [ACK] Seq=22 Ack=42 Win=10233 Len=0
29	257.483313	127.0.0.1	127.0.0.1	TCP	46	26441 → 9050 [PSH, ACK] Seq=42 Ack=22 Win=10233 Len=2
30	257.483358	127.0.0.1	127.0.0.1	TCP	44	9050 → 26441 [ACK] Seq=22 Ack=44 Win=10233 Len=0
31	257.638053	127.0.0.1	127.0.0.1	TCP	46	26441 → 9050 [PSH, ACK] Seq=44 Ack=22 Win=10233 Len=2
32	257.638101	127.0.0.1	127.0.0.1	TCP	44	9050 → 26441 [ACK] Seq=22 Ack=46 Win=10233 Len=0
33	257.799854	127.0.0.1	127.0.0.1	TCP	45	26441 → 9050 [PSH, ACK] Seq=46 Ack=22 Win=10233 Len=1
34	257.799904	127.0.0.1	127.0.0.1	TCP	44	9050 → 26441 [ACK] Seq=22 Ack=47 Win=10232 Len=0
35	257.986185	127.0.0.1	127.0.0.1	TCP	47	26441 → 9050 [PSH, ACK] Seq=47 Ack=22 Win=10233 Len=3
36	257.986235	127.0.0.1	127.0.0.1	TCP	44	9050 → 26441 [ACK] Seq=22 Ack=50 Win=10232 Len=0

> Frame 25: 50 bytes on wire (400 bits), 50 bytes captured (400 bits) on interface \Device\NPF_{loopback}, 0000 02 00 00 00 45 00 00 20 4c 18 40 00 00 06 00 00.....L@....> Null/loopback7f 00 00 01 7f 00 00 01 67 49 23 5a b9 b5 ef 3e.....gi#Z...> Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.10020 b9 79 96 f4 50 18 27 f9 d3 9c 00 00 67 65 69 61..y..P..geia> Transmission Control Protocol, Src Port: 26441, Dst Port: 9050, Seq: 34, Ack: 22, Len: 60030 61 61..aa> Data (6 bytes)

ΣΥΓΚΡΙΣΕΙΣ

1. TCP

Το πρωτόκολλο TCP είναι απλό και δεν παρέχει καμία ενσωματωμένη ασφάλεια. Οι πληροφορίες που αποστέλλονται μέσω TCP είναι εκτεθειμένες σε υποκλοπές και τροποποιήσεις από τρίτους. Είναι χρήσιμο για τη διασφάλιση αξιόπιστης μετάδοσης δεδομένων, αλλά δεν παρέχει κρυπτογράφηση.

2. TLS

Το TLS (Transport Layer Security) παρέχει κρυπτογράφηση και επαλήθευση ταυτότητας, καθιστώντας την επικοινωνία πολύ πιο ασφαλή από το απλό TCP. Με το TLS, τα δεδομένα κρυπτογραφούνται, προστατεύοντας τα από υποκλοπές και επιθέσεις. Αυτό είναι συνήθως το πρότυπο για ασφαλείς συνδέσεις (π.χ. HTTPS). Χρησιμοποιούνται ψηφιακά πιστοποιητικά για να πιστοποιούν τον server και τον client.

3. IPsec με ESP

Το IPsec (Internet Protocol Security) με το πρωτόκολλο ESP (Encapsulating Security Payload) παρέχει ασφάλεια σε επίπεδο IP. Κρυπτογραφεί το περιεχόμενο των πακέτων και εξασφαλίζει την ακεραιότητά τους. Είναι πιο κατάλληλο για VPN και άλλες εφαρμογές που απαιτούν ασφάλεια σε χαμηλότερο επίπεδο δικτύου, προσφέροντας προστασία από ευρύ φάσμα απειλών. Η πολυπλοκότητα και η ανάγκη για ειδική ρύθμιση των Security Associations μπορεί να είναι δύσκολη για μη έμπειρους χρήστες.

4. Δίκτυο Tor

Το Tor (The Onion Router) προσφέρει την υψηλότερη ανωνυμία και ασφάλεια από τις άλλες επιλογές. Χρησιμοποιεί πολλαπλά επίπεδα κρυπτογράφησης και αναδρομολογεί την κυκλοφορία μέσω διαφορετικών κόμβων, καθιστώντας δύσκολο να εντοπιστεί η πηγή της επικοινωνίας. Αν και είναι ασφαλές για ανώνυμη περιήγηση, ενδέχεται να είναι πιο αργό λόγω της πολυεπίπεδης αναδρομολόγησης.

Σύγκριση Ασφάλειας

1. **TCP**: Χρήσιμο για αξιόπιστη μετάδοση, αλλά δεν παρέχει καμία ασφάλεια.
2. **TLS**: Εύκολο στη χρήση και ιδανικό για προστασία εφαρμογών, καλύπτει την ασφάλεια των δεδομένων.
3. **IPsec/ESP**: Παρέχει την ισχυρότερη προστασία, αλλά είναι πιο δύσκολο στη ρύθμιση και ανάλυση.
4. **Tor**: Προσφέρει την ανώτερη ασφάλεια και ανωνυμία, ιδανικό για περιπτώσεις που η ιδιωτικότητα είναι κρίσιμη.

Συνολικά, το επίπεδο ασφάλειας αυξάνεται από το TCP προς το Tor. Ωστόσο, η επιλογή εξαρτάται από την εφαρμογή και τις απαιτήσεις για απόδοση και ιδιωτικότητα. Αν επιθυμούμε μέγιστη ασφάλεια, η επιλογή (IPsec - ESP) είναι η καλύτερη. Αν επιθυμούμε ευκολία ρύθμισης για εφαρμογές, το TLS/SSL είναι η πιο πρακτική λύση.